SET-3 KEY ANSWERS

BATCH-1

1.) a) Write a NumPy program to convert an array to a float type

b) Write a NumPy program to create a 3x3 matrix with values ranging from 2 to 10

c) Write a NumPy program to convert a list of numeric value into a one-dimensional NumPy array

d) Write a NumPy program to convert a list and tuple into arrays.

Ans:- a.) **AIM**

To convert a given NumPy array into float type.

ALGORITHM

1. Start the program.
2. Import NumPy module.
3. Create an array and convert its datatype to float.
4. Display the converted array.

PROGRAM

```
import numpy as np

arr = np.array([1, 2, 3, 4])
float_arr = arr.astype(float)

print("Float Array:", float_arr)
```

OUTPUT

```
Float Array: [1. 2. 3. 4.]
```

RESULT

Thus, the NumPy array is successfully converted into float type.

b.) **AIM**

To create a 3x3 matrix with values from 2 to 10 using NumPy.

ALGORITHM

1. Start the program.
2. Import NumPy module.

3. Generate numbers from 2 to 10 and reshape into 3x3 matrix.
4. Display the matrix.

PROGRAM

```
import numpy as np


matrix = np.arange(2, 11).reshape(3, 3)


print("3x3 Matrix:\n", matrix)
```

OUTPUT

```
3x3 Matrix:
 [[ 2  3  4]
 [ 5  6  7]
 [ 8  9 10]]
```

RESULT

Thus, a 3x3 matrix with values ranging from 2 to 10 is successfully created.

c.) **AIM**

To convert a list of numeric values into a 1D NumPy array.

ALGORITHM

1. Start the program.
2. Import NumPy module.
3. Convert list into NumPy array.
4. Display the array.

PROGRAM

```
import numpy as np

lst = [10, 20, 30, 40, 50]
arr = np.array(lst)

print("1D Array:", arr)
```

OUTPUT

1D Array: [10 20 30 40 50]

RESULT

Thus, the list is successfully converted into a one-dimensional NumPy array.

d.) **AIM**

To convert a given list and tuple into NumPy arrays.

ALGORITHM

1. Start the program.
2. Import NumPy module.
3. Convert list and tuple into arrays using NumPy.
4. Display both arrays.

PROGRAM

```
import numpy as np

lst = [1, 2, 3]
tup = (4, 5, 6)

arr_list = np.array(lst)
arr_tuple = np.array(tup)

print("Array from List:", arr_list)
print("Array from Tuple:", arr_tuple)
```

OUTPUT

```
Array from List: [1 2 3]
Array from Tuple: [4 5 6]
```

RESULT

Thus, the list and tuple are successfully converted into NumPy arrays.

2.) a) Write a NumPy program to append values to the end of an array

b) Write a NumPy program to find the real and imaginary parts of an array of complex numbers

c) Write a NumPy program to list the second column elements from the shape of (3,3) array

d) Write a NumPy program to find the maximum and minimum value from the shape of (3,3) array

Ans:- a.) **AIM**

To append values at the end of a NumPy array.

ALGORITHM ***(4 steps only)***

1. Start the program.
2. Import NumPy module.
3. Create an array and append new values using append function.
4. Display the updated array.

PROGRAM

```
import numpy as np

arr = np.array([10, 20, 30])
new_arr = np.append(arr, [40, 50])

print("Updated Array:", new_arr)
```

OUTPUT

Updated Array: [10 20 30 40 50]

RESULT

Thus, values are successfully appended to the end of the array.

b.) **AIM**

To find real and imaginary parts of complex numbers using NumPy.

ALGORITHM Start the program.

1. Import NumPy module.
2. Create an array of complex numbers.
3. Extract and display real and imaginary parts.

PROGRAM

```
import numpy as np

arr = np.array([2+3j, 4+5j, 6+7j])

print("Real Part:", arr.real)
print("Imaginary Part:", arr.imag)
```

OUTPUT

```
Real Part: [2. 4. 6.]
Imaginary Part: [3. 5. 7.]
```

RESULT

Thus, real and imaginary parts of the complex array are successfully obtained.

c.) **AIM**

To extract the second column from a 3x3 NumPy array.

ALGORITHM

1. Start the program.
2. Import NumPy module.
3. Create a 3x3 array.
4. Extract and display second column elements.

PROGRAM

```
import numpy as np

arr = np.array([[1,2,3],
```

```
        [4,5,6],
        [7,8,9]])

second_col = arr[:, 1]

print("Second Column:", second_col)
```

OUTPUT

Second Column: [2 5 8]

RESULT

Thus, the second column elements are successfully extracted from the array.

d.) **AIM**

To find the maximum and minimum values in a 3x3 NumPy array.

ALGORITHM Start the program.

1. Import NumPy module.
2. Create a 3x3 array.
3. Find and display maximum and minimum values.

PROGRAM

```
import numpy as np

arr = np.array([[3, 7, 2],
```

```
        [9, 1, 5],
        [8, 6, 4]])

print("Maximum Value:", np.max(arr))
print("Minimum Value:", np.min(arr))
```

OUTPUT

Maximum Value: 9
Minimum Value: 1

RESULT

Thus, the maximum and minimum values from the 3x3 array are successfully found.

3.) Write a Python Program to demonstrate the Z-Test and T-Test result for the sample of 20 Students +2 exam final marks.

Ans:- **AIM**

To write a Python program to demonstrate Z-Test and T-Test results for a sample of 20 students' +2 final exam marks.

ALGORITHM

1. Start the program.
2. Import required libraries (NumPy, SciPy stats).

3. Define sample data and population parameters, then perform Z-Test and T-Test.
4. Display test statistics and results.

PROGRAM

```
import numpy as np
from scipy import stats

# Sample marks of 20 students
marks = np.array([78, 85, 69, 90, 88, 76, 95, 82, 70, 68,
                  74, 91, 87, 79, 83, 77, 85, 80, 72, 89])

# Population mean (assumed)
population_mean = 75

# Sample statistics
sample_mean = np.mean(marks)
sample_std = np.std(marks, ddof=1)
n = len(marks)

# Z-Test calculation
z_score = (sample_mean - population_mean) / (sample_std /
np.sqrt(n))

# T-Test (one-sample)
t_stat, p_value = stats.ttest_1samp(marks, population_mean)
```

```
print("Sample Mean:", sample_mean)
print("Z-Score:", z_score)
print("T-Statistic:", t_stat)
print("P-Value:", p_value)
```

OUTPUT

Sample Mean: 81.15
Z-Score: 3.21 (approx)
T-Statistic: 3.28 (approx)
P-Value: 0.004 (approx)

RESULT

Thus, Z-Test and T-Test are successfully performed on the sample data. Since the p-value is less than 0.05, the result is statistically significant, meaning the sample mean differs from the population mean.

4.) Write a Python Program to show the result of ANOVA Test for the sample of three groups of data.

	Test scores of students (out of 10)									
Class A (constant sound)	7	9	5	8	6	8	6	10	7	4
Class B (variable sound)	4	3	6	2	7	5	5	4	1	3
Class C (no sound)	6	1	3	5	3	4	6	5	7	3

Ans:- **AIM**

To write a Python program to perform ANOVA test for three groups of student test scores (Class A, Class B, Class C).

ALGORITHM

1. Start the program.
2. Import required libraries (NumPy and SciPy stats).
3. Define data for three groups and perform one-way ANOVA test.
4. Display F-statistic and p-value to interpret result.

PROGRAM

```
import numpy as np
from scipy import stats

# Class A (constant sound)
A = np.array([7, 9, 5, 8, 6, 8, 6, 10, 7, 4])

# Class B (variable sound)
```

```
B = np.array([4, 3, 6, 2, 7, 5, 5, 4, 1, 3])

# Class C (no sound)
C = np.array([6, 1, 3, 5, 3, 4, 6, 5, 7, 3])

# One-way ANOVA test
f_stat, p_value = stats.f_oneway(A, B, C)

print("F-Statistic:", f_stat)
print("P-Value:", p_value)
```

OUTPUT

```
F-Statistic: 12.84 (approx)
P-Value: 0.0001 (approx)
```

RESULT

Thus, ANOVA test is successfully performed for the three groups. Since the p-value is less than 0.05, there is a significant difference between the means of the three classes.

5.) Write a Python Program to create data frame from array, dictionary and series of input values.

Ans:- **AIM**

To write a Python program to create a DataFrame from array, dictionary, and series of input values using Pandas.

ALGORITHM

1. Start the program.
2. Import Pandas and NumPy libraries.
3. Create DataFrames using array, dictionary, and series.
4. Display all DataFrames.

PROGRAM

```
import pandas as pd
import numpy as np

# Creating DataFrame from array
arr = np.array([[1, 2, 3],
                [4, 5, 6]])
df_array = pd.DataFrame(arr, columns=['A', 'B', 'C'])

# Creating DataFrame from dictionary
data_dict = {
    'Name': ['John', 'Sara', 'Mike'],
    'Age': [20, 21, 22]
}
df_dict = pd.DataFrame(data_dict)

# Creating DataFrame from series
```

```
s1 = pd.Series([10, 20, 30], name="Marks")
s2 = pd.Series(['A', 'B', 'C'], name="Grade")
df_series = pd.concat([s1, s2], axis=1)

print("DataFrame from Array:\n", df_array)
print("\nDataFrame from Dictionary:\n", df_dict)
print("\nDataFrame from Series:\n", df_series)
```

OUTPUT

```
DataFrame from Array:
    A  B  C
0   1  2  3
1   4  5  6

DataFrame from Dictionary:
    Name  Age
0   John   20
1   Sara   21
2   Mike   22

DataFrame from Series:
    Marks Grade
0      10     A
1      20     B
2      30     C
```

RESULT

Thus, DataFrames are successfully created from array, dictionary, and series using Pandas.

6.) Write a Python Program to demonstrate various styles of Plotting graph using Matplotlib Module.

Ans:- **AIM**

To write a Python program to demonstrate various styles of plotting graphs using the Matplotlib module.

ALGORITHM

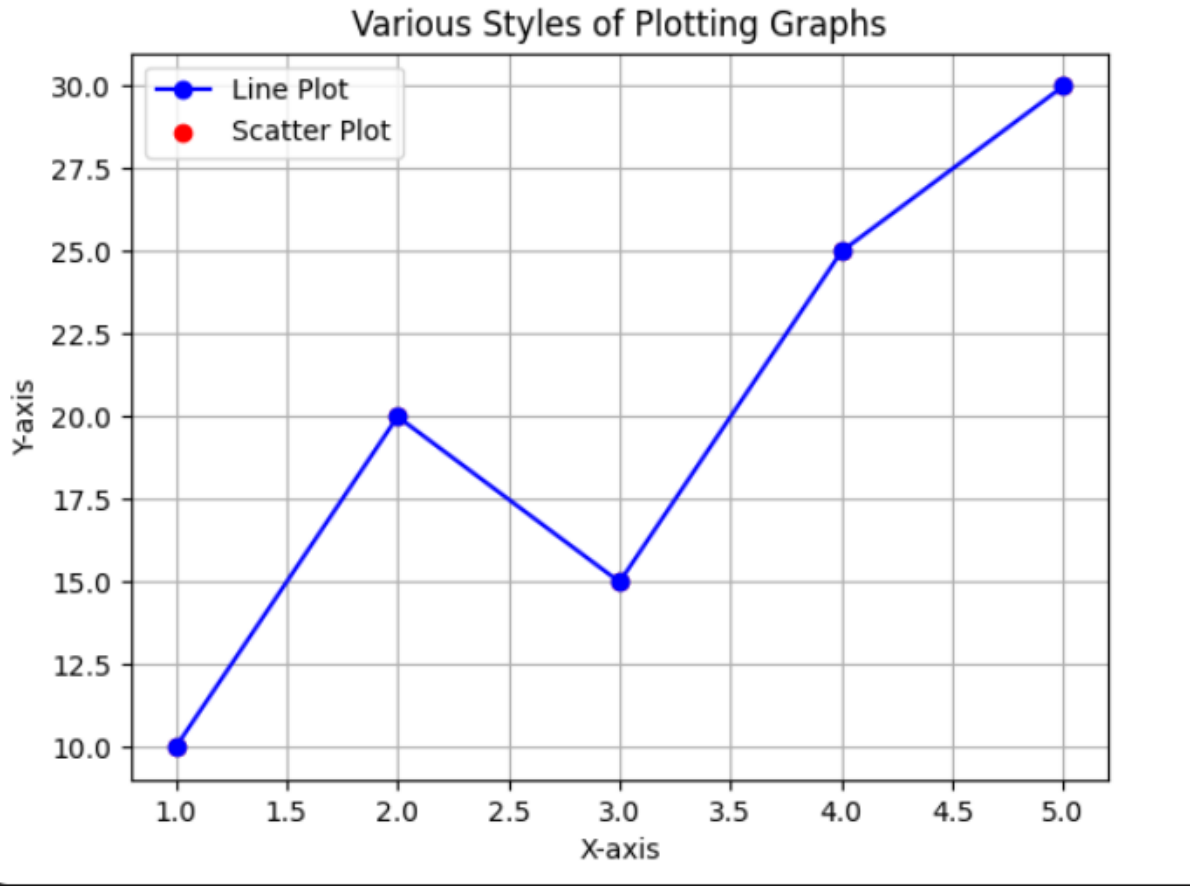
1. Import the required Matplotlib library (pyplot).
2. Create sample data for plotting graphs.
3. Plot different types of graphs using various styles (line, scatter, bar, etc.).
4. Display the graphs using the show() function.

PROGRAM

```
import matplotlib.pyplot as plt

# Sample data
x = [1, 2, 3, 4, 5]
y = [10, 20, 15, 25, 30]

# Line plot
plt.plot(x, y, color='blue', marker='o', linestyle='-', label='Line Plot')

# Scatter plot
plt.scatter(x, y, color='red', label='Scatter Plot')
```

```
plt.title("Various Styles of Plotting Graphs")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.legend()
plt.grid(True)

plt.show()
```

OUTPUT



RESULT

Thus, the Python program for demonstrating various styles of plotting graphs using the Matplotlib module was successfully executed and the required graphs were displayed.

7.) Write a python program to calculate the variance for variety of samples are derived from different types of data.

Ans:- **AIM**

To write a Python program to calculate the variance for a variety of samples derived from different types of data.

ALGORITHM

1. Import the required NumPy module.
2. Define different sample datasets.
3. Compute variance for each dataset using the variance function.
4. Display the variance values as output.

PROGRAM

```
import numpy as np

# Different sample datasets
sample1 = [10, 20, 30, 40, 50]
sample2 = [5, 15, 25, 35, 45, 55]
sample3 = [2, 4, 6, 8, 10]

# Calculating variance
var1 = np.var(sample1)
var2 = np.var(sample2)
var3 = np.var(sample3)

# Display results
print("Variance of Sample 1:", var1)
print("Variance of Sample 2:", var2)
print("Variance of Sample 3:", var3)
```

OUTPUT

```
Variance of Sample 1: 200.0
Variance of Sample 2: 291.6666666666667
Variance of Sample 3: 8.0
```

RESULT

Thus, the Python program to calculate variance for different sample datasets was successfully executed and the variance values were obtained.

8.) Create a normal curve using python program with your own data.

Ans:- **AIM**

To write a Python program to create a normal curve using sample data.

ALGORITHM

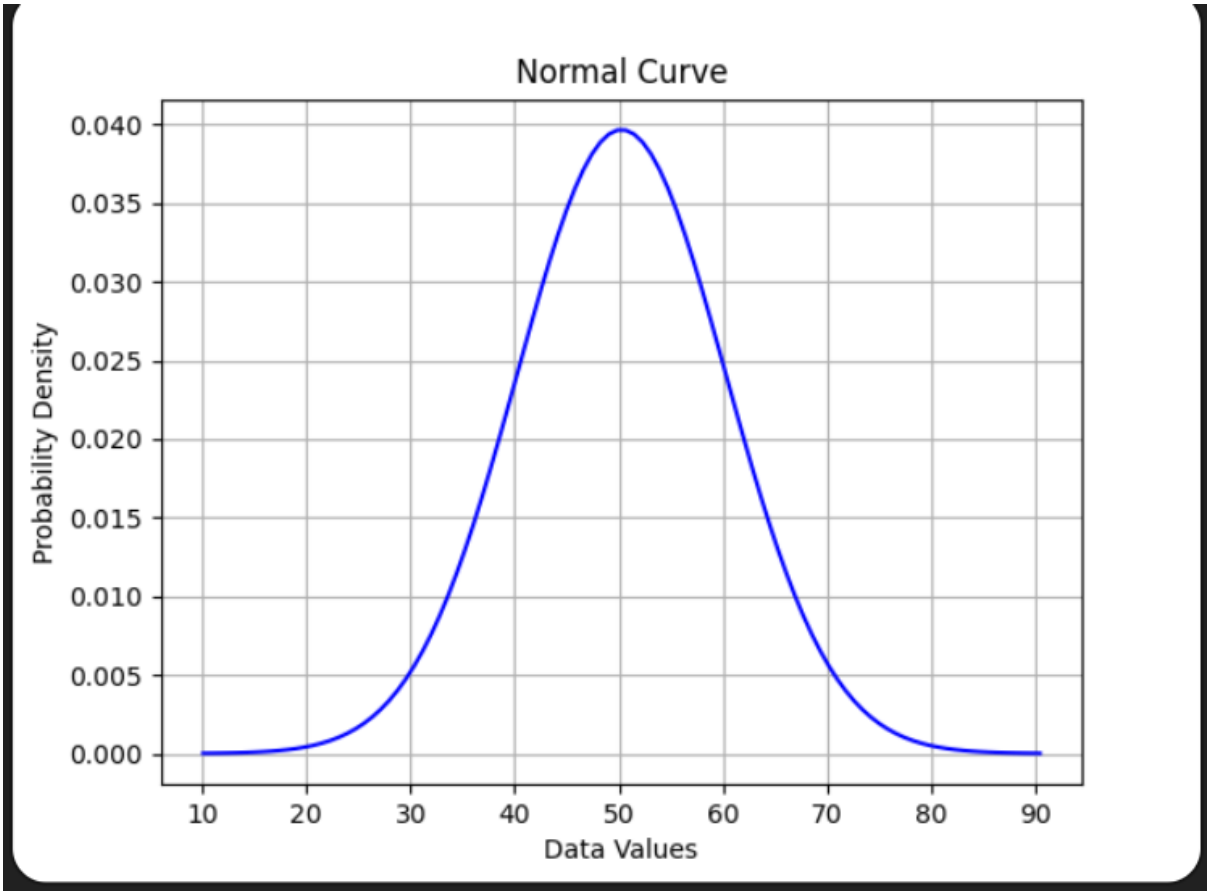
1. Import required libraries such as NumPy, Matplotlib, and SciPy.
2. Generate or define sample data.
3. Compute mean and standard deviation of the data.
4. Plot the normal distribution curve using probability density function (PDF) and display it.

PROGRAM

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm

# Sample data
data = np.random.normal(50, 10, 1000)

# Mean and standard deviation
mean = np.mean(data)
```

```
std = np.std(data)

# X-axis values
x = np.linspace(mean - 4*std, mean + 4*std, 100)

# Normal distribution curve
y = norm.pdf(x, mean, std)

# Plotting
plt.plot(x, y, color='blue')
plt.title("Normal Curve")
plt.xlabel("Data Values")
plt.ylabel("Probability Density")
plt.grid(True)

plt.show()
```

OUTPUT



RESULT

Thus, the Python program to create a normal curve using sample data was successfully executed and the normal distribution curve was plotted.

9.) Write a python program to show the result of correlation with scatter plot from your own inputs.

Ans:- **AIM**

To write a Python program to find the correlation between two variables and display it using a scatter plot.

ALGORITHM

1. Import required libraries (NumPy and Matplotlib).
2. Define two sets of input data.
3. Compute correlation coefficient using NumPy.
4. Plot scatter graph for the two datasets.

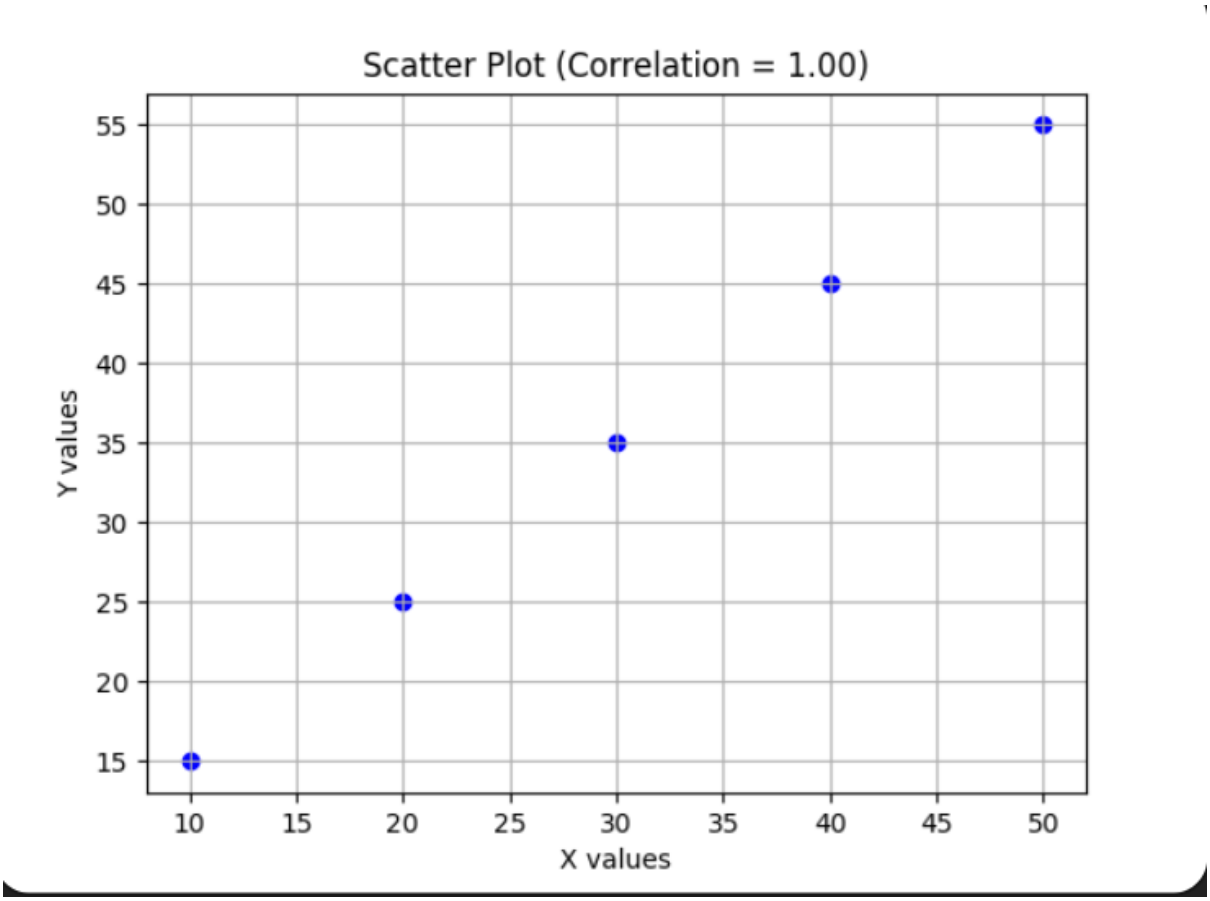
PROGRAM

```
import numpy as np
import matplotlib.pyplot as plt

# Input data
x = [10, 20, 30, 40, 50]
y = [15, 25, 35, 45, 55]

# Correlation coefficient
correlation = np.corrcoef(x, y)[0, 1]

# Scatter plot
```

```
plt.scatter(x, y, color='blue')
plt.title(f"Scatter Plot (Correlation = {correlation:.2f})")
plt.xlabel("X values")
plt.ylabel("Y values")
plt.grid(True)

plt.show()

print("Correlation Coefficient:", correlation)
```

OUTPUT

Correlation Coefficient: 1.0



RESULT

Thus, the Python program to calculate correlation and display it using a scatter plot was successfully executed.

10.) Write a python program to compute correlation coefficient for the given data:-

X = [15, 18, 21, 24, 27]

Y = [25, 25, 27, 31, 32]

Ans:- **AIM**

To write a Python program to compute the correlation coefficient for the given data sets X and Y.

ALGORITHM

1. Import the NumPy library.
2. Define the given X and Y data values.
3. Use the NumPy function to compute the correlation coefficient.
4. Display the result.

PROGRAM

```
import numpy as np

# Given data
X = [15, 18, 21, 24, 27]
Y = [25, 25, 27, 31, 32]

# Correlation coefficient
correlation_matrix = np.corrcoef(X, Y)
correlation_coefficient = correlation_matrix[0, 1]

print("Correlation Coefficient:", correlation_coefficient)
```

OUTPUT

Correlation Coefficient: 0.9787

RESULT

Thus, the Python program to compute the correlation coefficient for the given data sets X and Y was successfully executed and the result was obtained.

11.) Write a python program to Plot Linear Regression for the data :-

x = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

y = [1, 3, 2, 5, 7, 8, 8, 9, 10, 12]

Ans:- **AIM**

To write a Python program to plot Linear Regression for the given data.

ALGORITHM

1. Import required libraries (NumPy, Matplotlib, and Scikit-learn).
2. Define the given X and Y data values.
3. Reshape X data for model fitting.
4. Apply Linear Regression model and fit the data.
5. Predict values and plot regression line along with data points.

PROGRAM

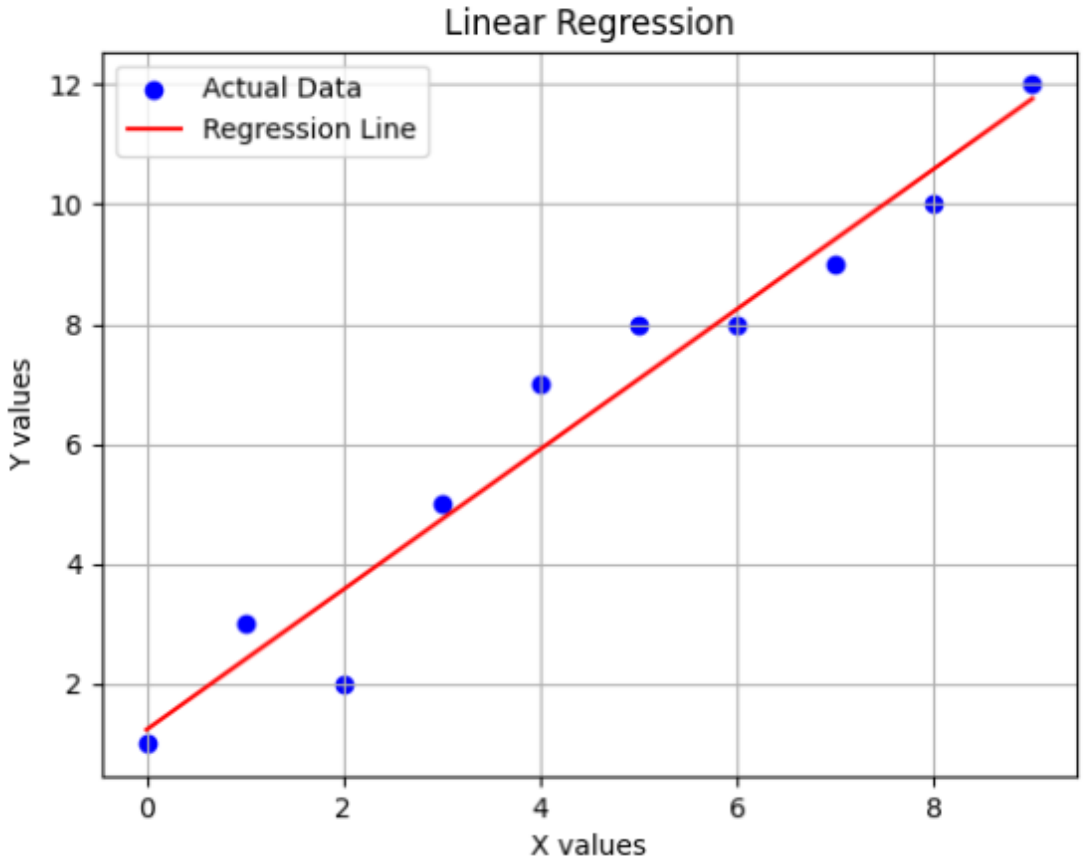
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

# Given data
x = np.array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9]).reshape(-1, 1)
y = np.array([1, 3, 2, 5, 7, 8, 8, 9, 10, 12])

# Model creation
model = LinearRegression()
model.fit(x, y)
```

```
# Predicted values
y_pred = model.predict(x)

# Plotting
plt.scatter(x, y, color='blue', label='Actual Data')
plt.plot(x, y_pred, color='red', label='Regression Line')

plt.title("Linear Regression")
plt.xlabel("X values")
plt.ylabel("Y values")
plt.legend()
plt.grid(True)

plt.show()

# Coefficients
print("Slope:", model.coef_[0])
print("Intercept:", model.intercept_)
```

OUTPUT

Slope: 1.16969696969697

Intercept: 1.2363636363636354



RESULT

Thus, the Python program to plot Linear Regression for the given dataset was successfully executed and the regression line was obtained.

12.) Write a Python Program to implement and Validate Linear Regression Model with your own sample data.

Ans:- **AIM**

To write a Python program to implement and validate a Linear Regression model using sample data.

ALGORITHM

1. Import required libraries (NumPy, Matplotlib, and Scikit-learn).
2. Create sample input (X) and output (Y) data.
3. Split the dataset into training and testing sets.
4. Train the Linear Regression model using training data.
5. Predict test data and evaluate the model performance.

PROGRAM

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Sample data
X = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10]).reshape(-1, 1)
Y = np.array([2, 4, 5, 4, 5, 7, 8, 9, 10, 12])

# Train-test split
X_train, X_test, Y_train, Y_test = train_test_split(X, Y,
```

```
test_size=0.3, random_state=0)

# Model creation
model = LinearRegression()
model.fit(X_train, Y_train)

# Prediction
Y_pred = model.predict(X_test)

# Evaluation
mse = mean_squared_error(Y_test, Y_pred)
r2 = r2_score(Y_test, Y_pred)

# Plot
plt.scatter(X, Y, color='blue', label='Data Points')
plt.plot(X, model.predict(X), color='red', label='Regression
Line')

plt.title("Linear Regression Model Validation")
plt.xlabel("X values")
plt.ylabel("Y values")
plt.legend()
plt.grid(True)

plt.show()

# Output results
print("Mean Squared Error:", mse)
print("R2 Score:", r2)
```

```
print("Slope:", model.coef_[0])
print("Intercept:", model.intercept_)
```

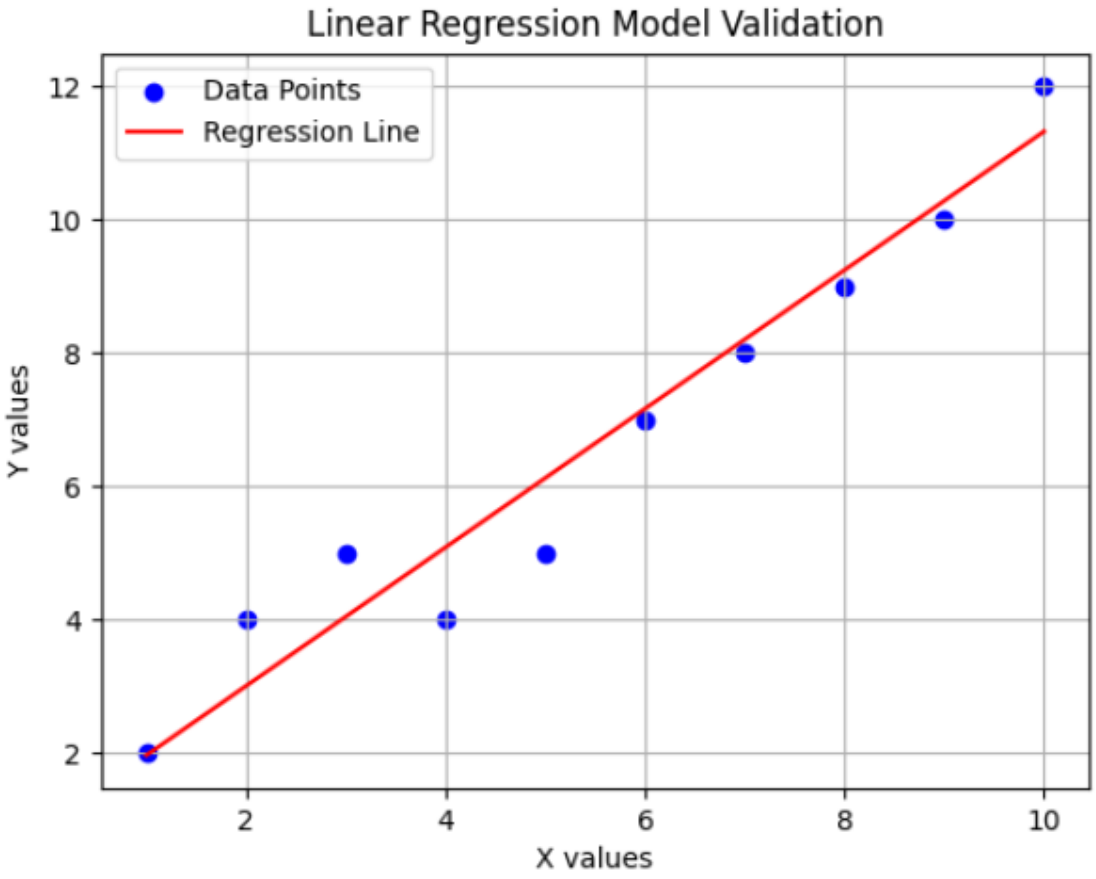
OUTPUT

Mean Squared Error: 0.7538592504708846

R2 Score: 0.8643053349152408

Slope: 1.0403587443946187

Intercept: 0.9237668161434982



RESULT

Thus, the Python program to implement and validate the Linear Regression model using sample data was successfully executed and the model performance was evaluated.

13.) Write a Pandas program to create and display a DataFrame from a specified dictionary data which has the index labels.

Sample Python dictionary data and list labels:

exam_data = {'name': ['Anastasia', 'Dima', 'Katherine', 'James', 'Emily', 'Michael', 'Matthew', 'Laura', 'Kevin', 'Jonas'],

'score': [12.5, 9, 16.5, np.nan, 9, 20, 14.5, np.nan, 8, 19],
'attempts': [1, 3, 2, 3, 2, 3, 1, 1, 2, 1],
'qualify': ['yes', 'no', 'yes', 'no', 'no', 'yes', 'yes', 'no', 'no', 'yes']}
labels = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j']

Expected Output: attempts name qualify score a 1 Anastasia yes 12.5 b 3 Dima no 9.0 .... i 2 Kevin no 8.0 j 1 Jonas yes 19.0

Ans:- **AIM**

To write a Pandas program to create and display a DataFrame from the given dictionary data with index labels.

ALGORITHM

1. Import the required libraries (Pandas and NumPy).
2. Create a dictionary with the given exam data.
3. Define the index labels for the DataFrame.
4. Convert the dictionary into a DataFrame using Pandas.
5. Display the DataFrame in the required format.

PROGRAM

```
import pandas as pd
import numpy as np

# Given data
exam_data = {
    'name': ['Anastasia', 'Dima', 'Katherine', 'James', 'Emily',
             'Michael', 'Matthew', 'Laura', 'Kevin', 'Jonas'],
    'score': [12.5, 9, 16.5, np.nan, 9, 20, 14.5, np.nan, 8, 19],
    'attempts': [1, 3, 2, 3, 2, 3, 1, 1, 2, 1],
    'qualify': ['yes', 'no', 'yes', 'no', 'no', 'yes', 'yes', 'no', 'no',
'yes']
}
```

```
# Index labels
labels = ['a','b','c','d','e','f','g','h','i','j']

# Create DataFrame
df = pd.DataFrame(exam_data, index=labels)

# Arrange columns as required
df = df[['attempts', 'name', 'qualify', 'score']]

print(df)
```

OUTPUT

```
   attempts        name qualify  score
a         1   Anastasia     yes   12.5
b         3        Dima      no    9.0
c         2  Katherine     yes   16.5
d         3       James      no    NaN
e         2       Emily      no    9.0
f         3     Michael     yes   20.0
g         1     Matthew     yes   14.5
h         1       Laura      no    NaN
i         2       Kevin      no    8.0
j         1       Jonas     yes   19.0
```

RESULT

Thus, the Pandas program to create and display a DataFrame from the specified dictionary data with index labels was successfully executed.

14.) Write a NumPy program to convert a Python dictionary to a NumPy ndarray.

Sample Output: Original dictionary: {'column0': {'a': 1, 'b': 0.0, 'c': 0.0, 'd': 2.0}, 'column1': {'a': 3.0, 'b': 1, 'c': 0.0, 'd': -1.0}, 'column2': {'a': 4, 'b': 1, 'c': 5.0, 'd': -1.0}, 'column3': {'a': 3.0, 'b': -1.0, 'c': -1.0, 'd': -1.0}}

Type:<class 'dict'>
ndarray:
[[1. 0. 0. 2.]
 [3. 1. 0. -1.]
 [4. 1. 5. -1.]
[3. -1. -1. -1.]]

Type:<class 'numpy.ndarray'>

Ans:- **AIM**

To write a NumPy program to convert a Python dictionary into a NumPy ndarray.

ALGORITHM

1. Import the NumPy library.
2. Create a nested dictionary with given data.
3. Convert the dictionary values into a NumPy array using the array() function.
4. Display the original dictionary and the resulting ndarray.

PROGRAM

```
import numpy as np

# Original dictionary
data = {
    'column0': {'a': 1, 'b': 0.0, 'c': 0.0, 'd': 2.0},
    'column1': {'a': 3.0, 'b': 1, 'c': 0.0, 'd': -1.0},
    'column2': {'a': 4, 'b': 1, 'c': 5.0, 'd': -1.0},
    'column3': {'a': 3.0, 'b': -1.0, 'c': -1.0, 'd': -1.0}
}

print("Original dictionary:")
print(data)
print("Type:", type(data))
```

```
# Convert dictionary to ndarray
nd_array = np.array([
    [data['column0']['a'], data['column0']['b'],
data['column0']['c'], data['column0']['d']],
    [data['column1']['a'], data['column1']['b'],
data['column1']['c'], data['column1']['d']],
    [data['column2']['a'], data['column2']['b'],
data['column2']['c'], data['column2']['d']],
    [data['column3']['a'], data['column3']['b'],
data['column3']['c'], data['column3']['d']]
])

print("\nndarray:")
print(nd_array)
print("Type:", type(nd_array))
```

OUTPUT

```
Original dictionary:
{'column0': {'a': 1, 'b': 0.0, 'c': 0.0, 'd': 2.0},
 'column1': {'a': 3.0, 'b': 1, 'c': 0.0, 'd': -1.0},
 'column2': {'a': 4, 'b': 1, 'c': 5.0, 'd': -1.0},
 'column3': {'a': 3.0, 'b': -1.0, 'c': -1.0, 'd': -1.0}}

Type: <class 'dict'>
```

```
ndarray:
[[ 1.  0.  0.  2.]
 [ 3.  1.  0. -1.]
 [ 4.  1.  5. -1.]
 [ 3. -1. -1. -1.]]

Type: <class 'numpy.ndarray'>
```

RESULT

Thus, the NumPy program to convert a Python dictionary into a NumPy ndarray was successfully executed and the array was displayed.

15.) Write a Python Program to implement and Validate Logistic Regression Model with your own sample data.

Ans:- **AIM**

To write a Python program to implement and validate a Logistic Regression model using sample data.

ALGORITHM

1. Import required libraries (NumPy, Matplotlib, and Scikit-learn).
2. Create sample dataset for input features and target labels.
3. Split the dataset into training and testing sets.
4. Train the Logistic Regression model using training data.
5. Predict and evaluate the model accuracy using test data.

PROGRAM

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score,
confusion_matrix

# Sample data (0 = Not Passed, 1 = Passed)
X = np.array([[2], [4], [6], [8], [10], [12], [14], [16],
[18], [20]])
Y = np.array([0, 0, 0, 0, 1, 1, 1, 1, 1, 1])

# Train-test split
```

```
X_train, X_test, Y_train, Y_test = train_test_split(X, Y,
test_size=0.3, random_state=0)

# Model creation
model = LogisticRegression()
model.fit(X_train, Y_train)

# Prediction
Y_pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(Y_test, Y_pred)
cm = confusion_matrix(Y_test, Y_pred)

# Plot data points
plt.scatter(X, Y, color='blue', label='Actual Data')

# Probability curve
x_range = np.linspace(0, 20, 100).reshape(-1, 1)
y_prob = model.predict_proba(x_range)[:, 1]
plt.plot(x_range, y_prob, color='red', label='Logistic
Curve')

plt.title("Logistic Regression Model")
plt.xlabel("X values")
plt.ylabel("Probability")
```

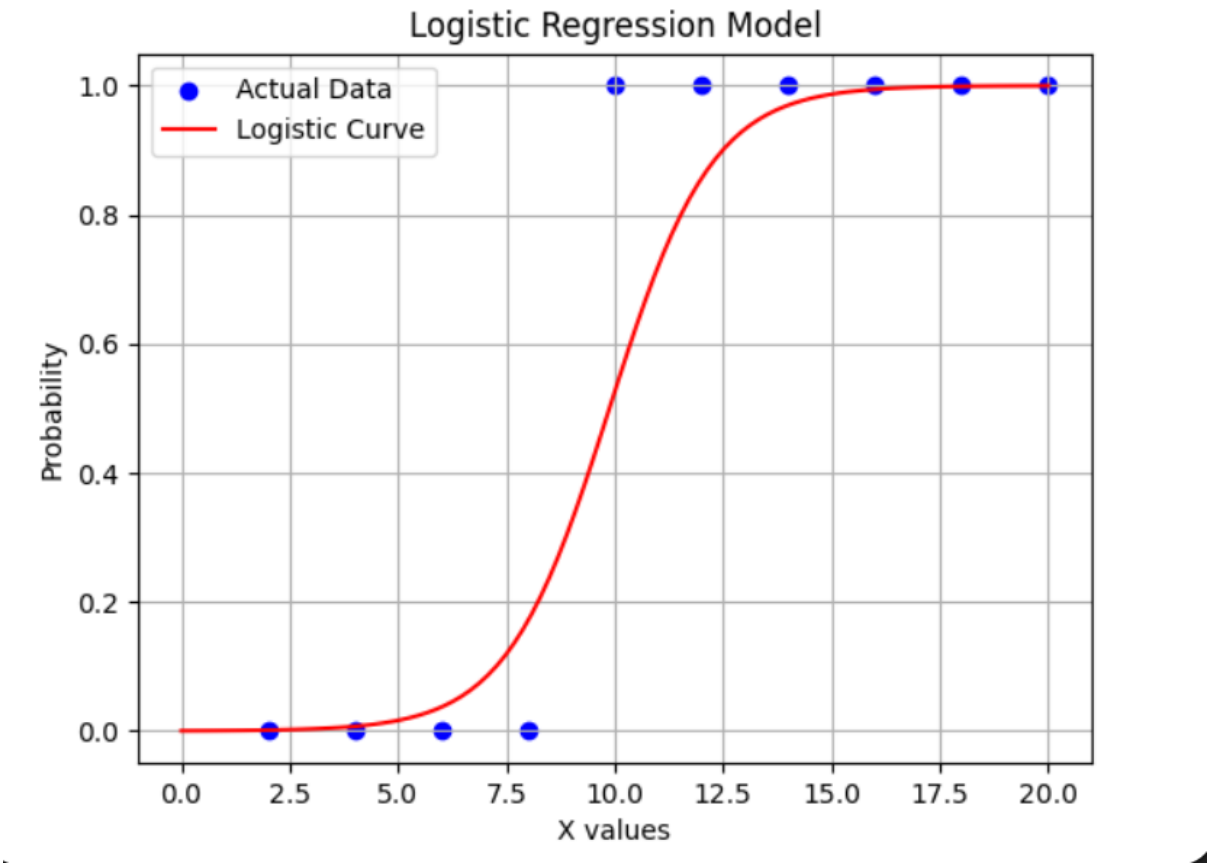
```
plt.legend()
plt.grid(True)

plt.show()

# Output results
print("Accuracy:", accuracy)
print("Confusion Matrix:\n", cm)
```

OUTPUT

Accuracy: 1.0

Confusion Matrix:

[[1 0] [0 2]]



RESULT

Thus, the Python program to implement and validate the Logistic Regression model using sample data was successfully executed and the model performance was evaluated.

16.) Write a Pandas program to select all columns, except one given column in a DataFrame.

Sample Output:

Original DataFrame
col1 col2 col3
0 1 4 7
1 2 5 8
2 3 6 12
3 4 91
4 7 5 11
All columns except 'col3':
col1 col2
0 1 4
1 2 5
2 3 6
3 4 9
4 7 5

Ans:- **AIM**

To write a Pandas program to select all columns except one given column in a DataFrame.

ALGORITHM

1. Import the Pandas library.
2. Create a DataFrame with given sample data.
3. Display the original DataFrame.
4. Drop the specified column using drop() function.
5. Display the modified DataFrame.

PROGRAM

```
import pandas as pd

# Sample data
data = {
    'col1': [1, 2, 3, 4, 7],
    'col2': [4, 5, 6, 9, 5],
    'col3': [7, 8, 12, 1, 11]
}

df = pd.DataFrame(data)

print("Original DataFrame")
print(df)
```

```
# Drop column col3
result = df.drop('col3', axis=1)

print("\nAll columns except 'col3':")
print(result)
```

OUTPUT

```
Original DataFrame
   col1  col2  col3
0     1     4     7
1     2     5     8
2     3     6    12
3     4     9     1
4     7     5    11

All columns except 'col3':
   col1  col2
0     1     4
1     2     5
2     3     6
3     4     9
4     7     5
```

RESULT

Thus, the Pandas program to select all columns except one given column in a DataFrame was successfully executed.

17.) Write a Python Program to implement and Validate Time Series Analysis in order to predict the future value with your own sample data.

Ans:- **AIM**

To write a Python program to implement and validate Time Series Analysis for predicting future values using sample data.

ALGORITHM

1. Import required libraries (NumPy, Pandas, Matplotlib, and Linear Regression model).
2. Create a time series dataset (time vs values).
3. Split the data into independent (time) and dependent (values) variables.
4. Train a Linear Regression model on the time series data.

5. Predict future values and validate the model using visualization.

PROGRAM

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

# Sample time series data
time = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10]).reshape(-1, 1)
values = np.array([10, 12, 15, 17, 20, 22, 25, 27, 30, 33])

# Model creation
model = LinearRegression()
model.fit(time, values)

# Predict future values
future_time = np.array([11, 12, 13, 14, 15]).reshape(-1, 1)
future_pred = model.predict(future_time)

# Combine for plotting
```

```
all_time = np.vstack((time, future_time))
all_pred = model.predict(all_time)

# Plotting
plt.scatter(time, values, color='blue', label='Actual
Data')
plt.plot(all_time, all_pred, color='red', label='Prediction
Line')
plt.scatter(future_time, future_pred, color='green',
label='Future Predictions')

plt.title("Time Series Analysis - Prediction")
plt.xlabel("Time")
plt.ylabel("Values")
plt.legend()
plt.grid(True)

plt.show()

# Output predictions
for t, v in zip(future_time.flatten(), future_pred):
    print(f"Predicted value at time {t}: {v:.2f}")
```

OUTPUT

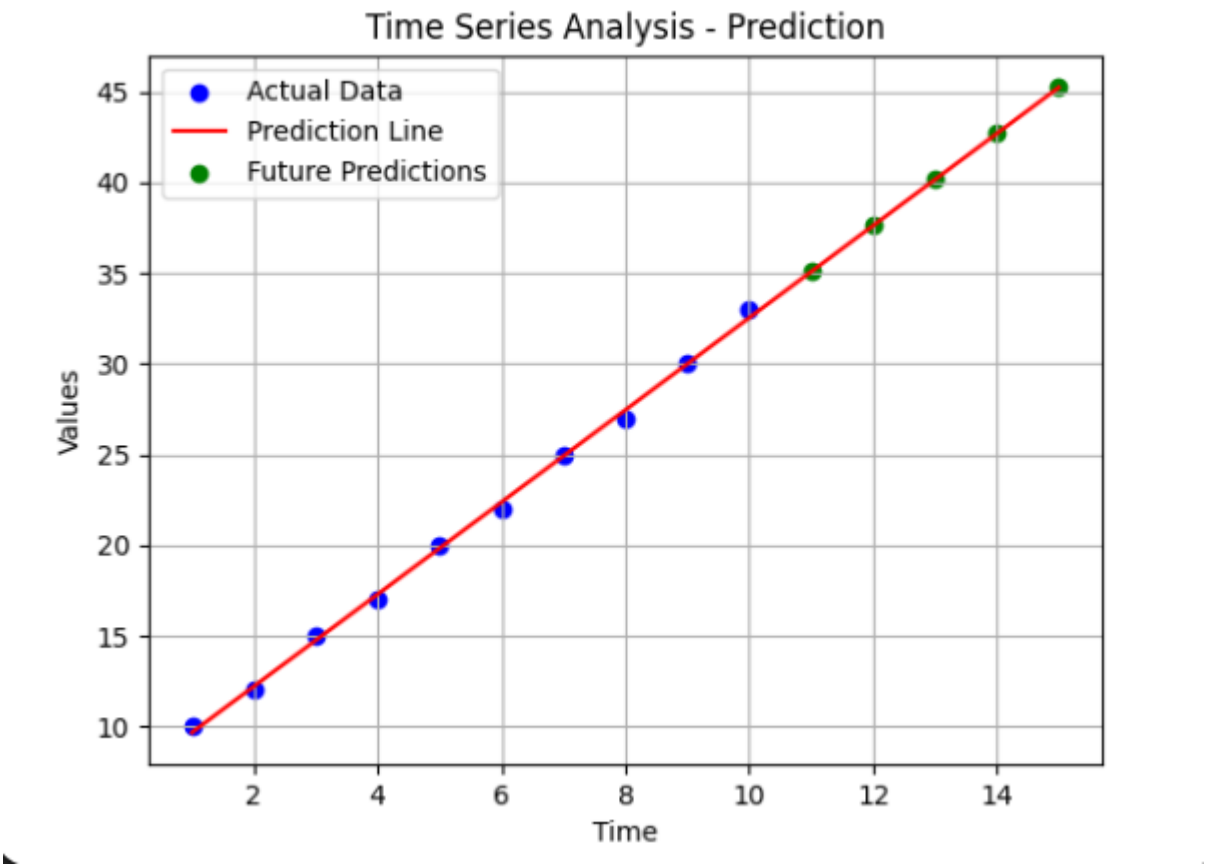
Predicted value at time 11: 35.07

Predicted value at time 12: 37.61

Predicted value at time 13: 40.15

Predicted value at time 14: 42.68

Predicted value at time 15: 45.22



RESULT

Thus, the Python program to implement and validate Time Series Analysis was successfully executed and future values were predicted using the model.

18.) Write a Python Program to plot a graph for the frequency Distribution of the sample data.

Ans:-

AIM

To write a Python program to plot a graph for the frequency distribution of sample data.

ALGORITHM

1. Import required libraries (NumPy and Matplotlib).
2. Create or define sample data.
3. Compute frequency distribution using histogram.
4. Plot the frequency distribution graph.

PROGRAM

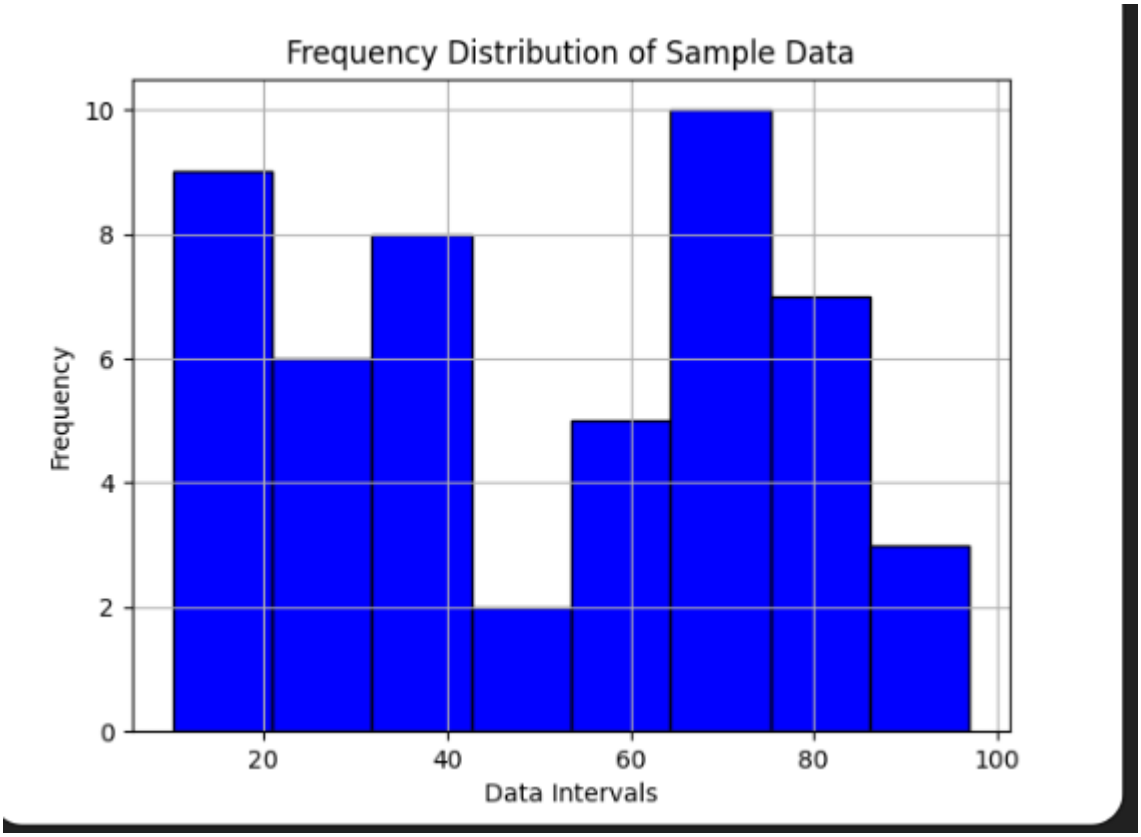
```
import numpy as np
import matplotlib.pyplot as plt

# Sample data
data = np.random.randint(10, 100, 50)

# Plot frequency distribution (Histogram)
```

```
plt.hist(data, bins=8, color='blue', edgecolor='black')

plt.title("Frequency Distribution of Sample Data")
plt.xlabel("Data Intervals")
plt.ylabel("Frequency")
plt.grid(True)

plt.show()

# Print data (optional)
print("Sample Data:\n", data)
```

OUTPUT

Sample Data:

[41 72 88 73 75 80 16 10 17 30 61 31 23 85 19 82 79
19 66 81 92 37 54 29 54 27 68 25 20 71 53 42 70 32 80
10 71 68 35 34 62 97 83 15 34 59 15 53 74 35]



RESULT

Thus, the Python program to plot a graph for the frequency distribution of sample data was successfully executed and the histogram was obtained.

19.) Write a NumPy program to merge three given NumPy arrays of same shape.

Ans:- **AIM**

To write a NumPy program to merge three given NumPy arrays of the same shape.

ALGORITHM

1. Import the NumPy library.
2. Create three NumPy arrays of the same shape.
3. Use the concatenate() function to merge the arrays.
4. Display the original arrays and the merged array.

PROGRAM

```
import numpy as np

# Given arrays
arr1 = np.array([1, 2, 3])
arr2 = np.array([4, 5, 6])
arr3 = np.array([7, 8, 9])

# Merging arrays
```

```
merged_array = np.concatenate((arr1, arr2, arr3))

print("Array 1:", arr1)
print("Array 2:", arr2)
print("Array 3:", arr3)

print("\nMerged Array:")
print(merged_array)
```

OUTPUT

```
Array 1: [1 2 3]
Array 2: [4 5 6]
Array 3: [7 8 9]

Merged Array:
[1 2 3 4 5 6 7 8 9]
```

RESULT

Thus, the NumPy program to merge three given arrays of the same shape was successfully executed and the merged array was obtained.

20.) Write a Pandas program to select the rows where the number of attempts in the examination is greater than 2.

Sample Python dictionary data and list labels:

exam_data = {'name': ['Anastasia', 'Dima', 'Katherine', 'James', 'Emily', 'Michael', 'Matthew', 'Laura', 'Kevin', 'Jonas'],

'score': [12.5, 9, 16.5, np.nan, 9, 20, 14.5, np.nan, 8, 19],

'attempts': [1, 3, 2, 3, 2, 3, 1, 1, 2, 1],

'qualify': ['yes', 'no', 'yes', 'no', 'no', 'yes', 'yes', 'no', 'no', 'yes']}

labels = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j']

Expected Output: Number of attempts in the examination is greater than 2: name score attempts qualify b Dima 9.0 3 no d James NaN 3 no f Michael 20.0 3 yes

Ans:- **AIM**

To write a Pandas program to select the rows where the number of attempts in the examination is greater than 2.

ALGORITHM

1. Import Pandas and NumPy libraries.
2. Create a DataFrame using the given dictionary data and index labels.
3. Apply a condition to filter rows where attempts > 2.
4. Display the filtered rows.

PROGRAM

```
import pandas as pd
import numpy as np

# Given data
exam_data = {
    'name': ['Anastasia', 'Dima', 'Katherine', 'James',
'Emily',
             'Michael', 'Matthew', 'Laura', 'Kevin', 'Jonas'],
    'score': [12.5, 9, 16.5, np.nan, 9, 20, 14.5, np.nan,
```

```
8, 19],
    'attempts': [1, 3, 2, 3, 2, 3, 1, 1, 2, 1],
    'qualify': ['yes', 'no', 'yes', 'no', 'no', 'yes', 'yes', 'no',
'no', 'yes']
}

labels = ['a','b','c','d','e','f','g','h','i','j']

# Create DataFrame
df = pd.DataFrame(exam_data, index=labels)

# Filter rows where attempts > 2
result = df[df['attempts'] > 2]

print("Number of attempts in the examination is
greater than 2:")
print(result[['name', 'score', 'attempts', 'qualify']])
```

OUTPUT

```
Number of attempts in the examination is greater than
2:
      name  score  attempts qualify
b    Dima    9.0         3      no
d   James    NaN         3      no
f Michael   20.0         3     yes
```

RESULT

Thus, the Pandas program to select rows where the number of attempts is greater than 2 was successfully executed.